



**OFICINA DE SOLUÇÕES
PROFISSIONAIS**

ORIENTAÇÕES

OFICINA DE SOLUÇÕES PROFISSIONAIS

Disciplina: Segurança e Auditoria de Sistemas
Unidade: 3- Segurança na Internet, Dispositivos Móveis e Testes de Intrusão

Encontro 3 – Simulação de Situações Profissionais

Neste encontro, você assumirá um papel específico em uma situação que visa desenvolver competências profissionais.

Você deverá concluir a atividade e apresentá-la ao final do encontro.

1. Objetivo da Atividade

Identificar vulnerabilidades em uma aplicação web e propor a aplicação de técnicas de proteção contra ataques de injeção, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) e outras ameaças comuns à segurança na internet.

2. Descrição da Atividade

A empresa EduConnect desenvolveu uma plataforma web para gerenciamento de cursos online. A aplicação permite que alunos se cadastrem, façam login, publiquem comentários nos fóruns, enviem arquivos de trabalhos e acessem suas notas via uma API. Durante uma auditoria interna, foram identificados os seguintes problemas: o campo de login realiza consultas SQL sem validação da entrada do usuário; os comentários do fórum exibem o texto digitado diretamente na página sem nenhum tratamento; o formulário de exclusão de conta não possui mecanismo de verificação de origem da solicitação; as senhas aceitas pelo sistema não possuem requisitos mínimos de complexidade; o upload de arquivos aceita qualquer extensão sem restrição; e a API de notas pode ser acessada sem autenticação. A equipe de desenvolvimento foi acionada para corrigir as falhas antes do lançamento público da plataforma.

- SQL Injection: ocorre quando entradas do usuário são inseridas diretamente em consultas SQL sem validação, permitindo ao atacante manipular o banco de dados.
- Cross-Site Scripting (XSS): ocorre quando dados fornecidos pelo usuário são exibidos na página sem tratamento, permitindo a execução de scripts maliciosos no navegador de outras pessoas.
- Cross-Site Request Forgery (CSRF): ocorre quando uma solicitação maliciosa é enviada em nome de um usuário autenticado sem o seu conhecimento, explorando a confiança da aplicação na sessão ativa.
- Outras vulnerabilidades: senhas fracas, upload irrestrito de arquivos e APIs sem autenticação são vetores igualmente explorados em ataques reais a aplicações web.

Para responder às questões, considere as técnicas de proteção apresentadas a seguir e relacione cada uma às vulnerabilidades identificadas no cenário:

- Consultas parametrizadas (Prepared Statements): separam o comando SQL dos dados fornecidos pelo usuário, impedindo a injeção de código.
- Escape de HTML/JavaScript e Content Security Policy (CSP): tratam os dados antes de exibi-los na página, impedindo a execução de scripts maliciosos.
- Tokens anti-CSRF e autenticação obrigatória: verificam a legitimidade das requisições e impedem que ações sensíveis sejam executadas sem consentimento do usuário autenticado.
- Políticas de senha, restrição de upload e autenticação de API: complementam a proteção da aplicação contra ataques de força bruta, execução de arquivos maliciosos e acessos não autorizados a recursos.

Com base no cenário da EduConnect e nas técnicas apresentadas, responda às questões a seguir:

Identifique cada vulnerabilidade presente no cenário da EduConnect e indique a qual tipo de ataque ela corresponde (SQL Injection, XSS, CSRF, força bruta, upload inseguro ou API sem autenticação);

Para cada vulnerabilidade identificada, descreva a técnica de proteção que deve ser aplicada, explicando como ela funciona e por que é eficaz contra aquele tipo de ataque;

Justifique por que a segurança de aplicações web é considerada um processo contínuo e não uma solução única e definitiva

3. Etapas para Realização

Leia o cenário da EduConnect com atenção e, para cada problema descrito, anote em um rascunho: o que está errado, onde está errado e o que poderia acontecer se um atacante explorasse essa falha.

Consulte as técnicas de proteção apresentadas no material da aula e associe cada vulnerabilidade identificada à sua respectiva técnica. Por exemplo: se o problema é que o campo de login não valida a entrada do usuário antes de montar a consulta SQL, a técnica correspondente é o uso de Prepared Statements.

Para cada associação feita, explique com suas palavras como a técnica age sobre o problema — não basta nomear a proteção, é preciso demonstrar que você entende o mecanismo. Por exemplo: "o Prepared Statement separa o comando SQL dos dados do usuário, de modo que mesmo que o usuário digite ' OR '1'='1, esse trecho será tratado como dado e não como parte do comando".

Escolha duas das técnicas trabalhadas e escreva um exemplo prático de como elas seriam aplicadas no código da EduConnect. Não é necessário código funcional completo — um

pseudocódigo ou trecho comentado já é suficiente. Sugestões de técnicas para exemplificar: Prepared Statement no login, escape de HTML nos comentários do fórum, token anti-CSRF no formulário de exclusão de conta, ou autenticação com token na API de notas.

Por último, reflita sobre a questão da segurança contínua. Pense: as técnicas aplicadas hoje resolvem os problemas de hoje — mas e amanhã? Considere aspectos como surgimento de novas vulnerabilidades, atualização de bibliotecas, testes periódicos e o papel do OWASP Top 10 como referência de ameaças recorrentes.

GABARITO:

Olá, Tutor (a). O aluno(a) deverá desenvolver um relatório/identificação de acordo com o problema proposto. Como resultado esperado, teremos algo parecido como:

Questão 1 — Identificação das vulnerabilidades e tipos de ataque

Vulnerabilidade no cenário	Tipo de ataque
Campo de login com consulta SQL sem validação	SQL Injection
Comentários do fórum exibidos sem tratamento na página	Cross-Site Scripting (XSS)
Formulário de exclusão de conta sem verificação de origem	Cross-Site Request Forgery (CSRF)
Senhas sem requisitos mínimos de complexidade	Ataque de Força Bruta
Upload de arquivos sem restrição de tipo ou tamanho	Upload inseguro de arquivos
API de notas acessível sem autenticação	Acesso não autorizado a API

Questão 2 — Técnica de proteção e funcionamento

SQL Injection → Prepared Statements (consultas parametrizadas) O comando SQL é definido primeiro, com marcadores de posição para os valores. Os dados fornecidos pelo usuário são enviados separadamente e tratados como parâmetro, nunca como parte do comando. Assim,

mesmo que o usuário digite ' OR '1'=1, esse texto é interpretado como dado e não altera a lógica da consulta.

XSS → Escape de HTML e Content Security Policy (CSP) Antes de exibir qualquer texto digitado pelo usuário na página, todos os caracteres especiais de HTML são convertidos para suas entidades seguras — por exemplo, < vira <. Com isso, um script digitado no fórum é exibido como texto na tela e nunca executado pelo navegador. O CSP complementa essa proteção indicando ao navegador de quais origens scripts podem ser carregados.

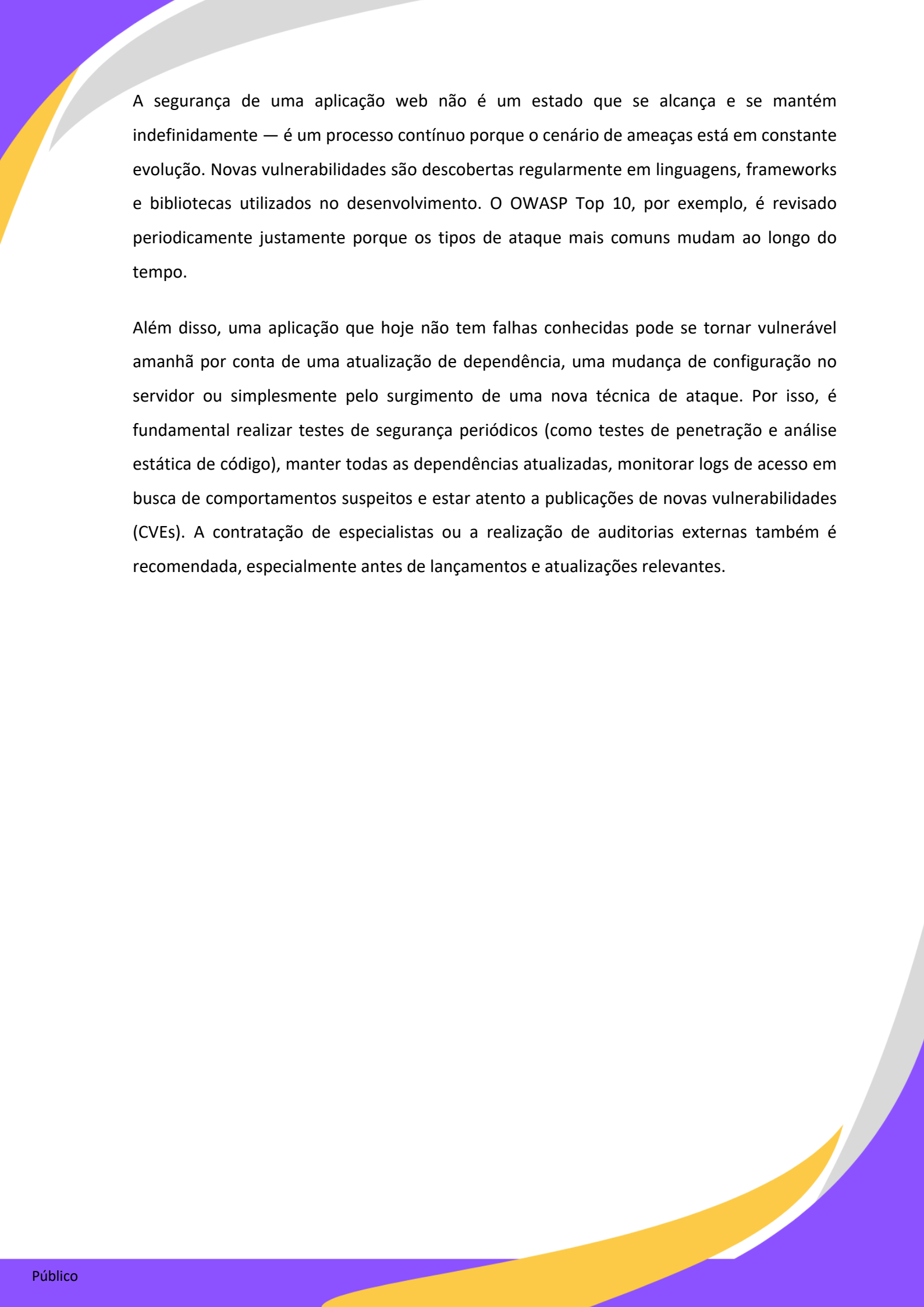
CSRF → Token anti-CSRF Um valor único e imprevisível é gerado pelo servidor e incluído em cada formulário sensível. Quando o formulário é submetido, o servidor verifica se o token enviado corresponde ao token da sessão. Uma requisição forjada por um atacante externo não terá esse token válido e será rejeitada.

Força bruta → Política de senhas fortes + bloqueio de conta Exigir senhas com letras maiúsculas, minúsculas, números e símbolos aumenta exponencialmente o número de combinações possíveis, tornando ataques de força bruta inviáveis na prática. O bloqueio temporário após um número definido de tentativas fracassadas (por exemplo, 5 tentativas) impede varreduras automatizadas.

Upload inseguro → Validação de tipo, extensão e conteúdo A aplicação deve verificar tanto a extensão do arquivo quanto o conteúdo real do arquivo (MIME type), além de impor um tamanho máximo. Arquivos com extensões executáveis (.php, .exe, .sh) devem ser bloqueados. O armazenamento deve ocorrer fora do diretório público do servidor.

API sem autenticação → Tokens de acesso (ex.: JWT) com autorização Toda requisição à API deve incluir um token de acesso válido gerado no momento do login. O servidor valida o token antes de retornar qualquer dado. Tokens de atualização permitem renovar o acesso sem exigir novo login, mantendo a segurança sem comprometer a usabilidade.

Questão 3 — A natureza contínua da segurança de aplicações web



A segurança de uma aplicação web não é um estado que se alcança e se mantém indefinidamente — é um processo contínuo porque o cenário de ameaças está em constante evolução. Novas vulnerabilidades são descobertas regularmente em linguagens, frameworks e bibliotecas utilizados no desenvolvimento. O OWASP Top 10, por exemplo, é revisado periodicamente justamente porque os tipos de ataque mais comuns mudam ao longo do tempo.

Além disso, uma aplicação que hoje não tem falhas conhecidas pode se tornar vulnerável amanhã por conta de uma atualização de dependência, uma mudança de configuração no servidor ou simplesmente pelo surgimento de uma nova técnica de ataque. Por isso, é fundamental realizar testes de segurança periódicos (como testes de penetração e análise estática de código), manter todas as dependências atualizadas, monitorar logs de acesso em busca de comportamentos suspeitos e estar atento a publicações de novas vulnerabilidades (CVEs). A contratação de especialistas ou a realização de auditorias externas também é recomendada, especialmente antes de lançamentos e atualizações relevantes.